

PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE (IOHE)
ON
Performance Analysis of JVM Garbage Collectors under Memory-Intensive
Workloads
submitted in partial fulfilment of the requirements for the award of degree of
BACHELOR OF ENGINEERING
In
COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Priyanshi Jain

2210992102

Rakshita

2210992136

Taniya

2210992447

Divyanjali Singla

2210990296

Supervised By:

Rajat Takkar

Assistant Professor

Department of CSE

Chitkara University Institute of Engineering and Technology



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY

CONTENTS

S.No.	Title	Page No.
1	Acknowledgement	4
2	Abstract	5
3	Introduction	6
4	Methodology	7
5	Implementation	7
6	Tools and Technologies	8
7	Major Findings/Outcomes/Output/Results	8
8	Conclusion and Future Scope	13
9	References	14

DECLARATION

We hereby certify that the work which is being presented in the project report entitled “**Performance Analysis of JVM Garbage Collectors under Memory-Intensive Workloads**” in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of **Rajat Takkar**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date: 16/05/2026

**Priyanshi Jain, Rakshita,
Taniya, Divyanjali Singla**

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Rajat Takkar

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported us throughout the development of this research project. Their guidance, encouragement, and valuable feedback played an important role in the successful completion of this work. We are deeply thankful to our project mentor, Rajat Takkar, Assistant Professor, Department of Computer Science and Engineering, Chitkara University, for his constant support, expert guidance, and insightful suggestions during every phase of the project. His mentorship helped us improve both the technical and analytical aspects of this research. We would also like to thank the Department of Computer Science and Engineering, Chitkara University Institute of Engineering and Technology, for providing us with the learning environment and technical support required to complete this research successfully.

ABSTRACT

Memory management is a critical aspect of Java application performance. The Java Virtual Machine (JVM) provides various Garbage Collection (GC) algorithms, each optimized for different workloads and performance goals. This project presents a performance analysis of three primary JVM Garbage Collectors—Serial GC, Parallel GC, and G1 GC—under memory-intensive workloads. By executing a custom benchmarking suite (GCResearchBenchmark) with a fixed 2GB heap size (-Xms2g -Xmx2g), we measured critical metrics including Execution Time, Average Pause Time, Throughput, and GC Time. The results demonstrate how each algorithm manages object allocation and reclamation, highlighting the trade-offs between throughput-optimized collectors (Parallel GC) and latency-optimized collectors (G1 GC). This analysis serves as a guide for selecting the appropriate GC algorithm based on specific application requirements.

1. INTRODUCTION

1.1 Project Overview

The Java Virtual Machine (JVM) abstracts manual memory management through Garbage Collection (GC), a process that automatically reclaims memory occupied by unreachable objects. However, GC operations often require pausing application threads (Stop-The-World events), which can impact application latency and throughput. As enterprise applications scale and handle larger datasets, understanding the behavioral nuances of different GC algorithms becomes crucial. This research focuses on evaluating the performance of three widely used JVM Garbage Collectors:

1.2 Problem Statement

Selecting the wrong Garbage Collector for a memory-intensive Java application can lead to severe performance bottlenecks, including high CPU utilization overhead, extended application pauses, and reduced overall throughput. Developers often lack empirical data on how different collectors perform under high-allocation rates within constrained heap environments. This project aims to bridge that gap by conducting a direct, metric-driven comparison.

1.3 Objective

- To design and execute a benchmark (GCResearchBenchmark) simulating memory-intensive object allocation.
- To evaluate the Serial, Parallel, and G1 Garbage Collectors under identical heap configurations (2GB).
- To measure and analyze key performance indicators: Execution Time, Average Pause Time, Application Throughput, and CPU Usage.
- **Serial GC:** A simple, single-threaded collector suitable for small applications.
- **Parallel GC:** A multi-threaded collector designed to maximize throughput by utilizing multiple CPU cores for GC tasks.
- **G1 GC (Garbage-First):** A server-style collector targeted at multi-processor machines with large memories, designed to provide a balance between throughput and predictable pause times.

2. METHODOLOGY

The research was conducted using a structured empirical approach. We developed a custom Java application named GCResearchBenchmark designed to simulate a memory-intensive workload by rapidly allocating 256 KB blocks of memory within nested loops.

2.1 Benchmark Design

- **Warm-up Rounds:** 5 rounds to allow the JVM's Just-In-Time (JIT) compiler to optimize the code path before recording metrics.
- **Measured Rounds:** 15 rounds of active data collection.
- **Workload:** 120 outer loops and 80 inner allocations per round, generating significant memory churn to trigger frequent garbage collection cycles.

2.2 Execution Parameters

To ensure a fair comparison, the JVM heap size was locked to 2 Gigabytes for both initial and maximum sizes to prevent dynamic heap resizing overhead during the benchmark.

- **JVM Flags used:** -Xms2g -Xmx2g

2.3 Metrics Collected

- **Execution Time:** Total time taken to complete the benchmark rounds.
- **Avg Pause Time:** The average duration the application threads were halted for garbage collection.
- **Throughput:** The percentage of total execution time spent running application code versus garbage collection.
- **CPU Usage:** The CPU overhead incurred during the process.
- **Memory Before/After:** Heap memory footprint before and after the GC cycles.

3. IMPLEMENTATION

The implementation phase involved executing the GCResearchBenchmark compiled Java class from the command line, iteratively passing different GC flags to the JVM.

Execution Commands:

1. **Parallel GC Test:** `java -Xms2g -Xmx2g -XX:+UseParallelGC GCResearchBenchmark`
2. **Serial GC Test:** `java -Xms2g -Xmx2g -XX:+UseSerialGC GCResearchBenchmark`
3. **G1 GC Test:** `java -Xms2g -Xmx2g -XX:+UseG1GC GCResearchBenchmark`

The benchmark suite automatically handles the 5 warm-up rounds before initializing the metric counters for the 15 measured rounds. Real-time telemetry was output to the console, capturing GC Count Deltas, GC Time Deltas, and memory fluctuations per round.

4. TOOLS AND TECHNOLOGIES

To ensure accurate and reproducible results, the following tools and technologies were utilized:

- **Java Development Kit (JDK):** The core runtime environment. The specific JVM implementations of the Garbage Collectors were tested using standard JDK distributions.
- **Garbage Collectors Tested:**
 - -XX:+UseSerialGC
 - -XX:+UseParallelGC
 - -XX:+UseG1GC
- **Development Environment:** Visual Studio Code (VS Code) was used as the primary IDE for writing the benchmarking scripts.
- **Command-Line Interface:** Windows PowerShell was used to execute the benchmark commands and capture standard output streams for analysis.
- **Hardware Environment:** The tests were executed on a multi-core processor machine to adequately test the multi-threading capabilities of Parallel and G1 GCs.

5. MAJOR FINDINGS / RESULTS

The benchmarking process yielded distinct performance profiles for each Garbage Collector. Below are the execution outputs and our analysis.

5.1 Parallel GC Performance

When executed with -XX:+UseParallelGC, the collector prioritized raw throughput.

- **Round 1 Execution Time:** 256 ms
- **Throughput:** ~92.58% to 93.39%
- **Average Pause Time:** ~3.75 ms to 3.80 ms

Observation: Parallel GC efficiently utilized multiple threads to quickly reclaim memory, resulting in solid throughput and fast execution times for the memory blocks allocated.

```

PS C:\Users\jpry\OneDrive\Pictures\Desktop\VS Code\CC File> java -Xms2g -Xmx2g -XX:+UseParallelGC GCResearchBenchmark
=====
GC Research Benchmark
Warm-up Rounds : 5
Measured Rounds : 15
Outer Loops : 120
Inner Allocations: 80
Block Size : 256 KB
=====
Warm-up Round 1 completed
Warm-up Round 2 completed
Warm-up Round 3 completed
Warm-up Round 4 completed
Warm-up Round 5 completed
-----
Round : 1
Execution Time : 256 ms
Avg Pause Time : 3.80 ms
Throughput : 92.58 %
CPU Usage : 7.23 %
GC Count Delta : 5
GC Time Delta : 19 ms
Memory Before : 571.38 MB
Memory After : 276.39 MB
Checksum : 275424
-----
Round : 2
Execution Time : 227 ms
Avg Pause Time : 3.75 ms
Throughput : 93.39 %
CPU Usage : 6.00 %
GC Count Delta : 4
GC Time Delta : 15 ms

```

5.2 Serial GC Performance

When executed with `-XX:+UseSerialGC`, the collector operated on a single thread.

- **Round 1 Execution Time:** 205 ms
- **Throughput:** ~87.61% to 87.80%
- **Average Pause Time:** ~5.40 ms to 6.25 ms

Observation: As expected for a multi-core system running a single-threaded collector, the Serial GC showed the lowest overall application throughput. The application spent significantly more relative time waiting for the single GC thread to clean up the 256 KB memory blocks.

```

PS C:\Users\jpriy\OneDrive\Pictures\Desktop\VS Code\CC File> java -Xms2g -Xmx2g -XX:+UseSerialGC GCResearchBenchmark
=====
GC Research Benchmark
Warm-up Rounds   : 5
Measured Rounds  : 15
Outer Loops      : 120
Inner Allocations: 80
Block Size       : 256 KB
=====
Warm-up Round 1 completed
Warm-up Round 2 completed
Warm-up Round 3 completed
Warm-up Round 4 completed
Warm-up Round 5 completed
-----
Round           : 1
Execution Time  : 205 ms
Avg Pause Time  : 6.25 ms
Throughput      : 87.80 %
CPU Usage       : 6.18 %
GC Count Delta  : 4
GC Time Delta   : 25 ms
Memory Before   : 147.86 MB
Memory After    : 382.23 MB
Checksum        : 275424
-----
Round           : 2
Execution Time  : 218 ms
Avg Pause Time  : 5.40 ms
Throughput      : 87.61 %
CPU Usage       : 5.81 %

```

5.3 G1 GC (Garbage-First) Performance

When executed with `-XX:+UseG1GC`, the collector demonstrated its capability to balance pause times and throughput.

- **Round 1 & 2 Execution Times:** 206 ms, 191 ms
- **Throughput:** ~92.67% to 93.69%
- **Average Pause Time:** ~4.33 ms to 4.67 ms

Observation: G1 GC maintained an excellent throughput rivaling Parallel GC while managing the heap in regions. This allowed it to keep pause times relatively low and predictable, validating its design as a low-latency, high-throughput collector for modern workloads.

```

=====
PS C:\Users\jpriy\OneDrive\Pictures\Desktop\VS Code\CC File> java -Xms2g -Xmx2g -XX:+UseG1GC GCResearchBenchmark
=====
GC Research Benchmark
Warm-up Rounds   : 5
Measured Rounds  : 15
Outer Loops      : 120
Inner Allocations: 80
Block Size       : 256 KB
=====
Warm-up Round 1 completed
Warm-up Round 2 completed
Warm-up Round 3 completed
Warm-up Round 4 completed
Warm-up Round 5 completed
-----
Round           : 1
Execution Time  : 206 ms
Avg Pause Time  : 4.33 ms
Throughput      : 93.69 %
CPU Usage       : 7.08 %
GC Count Delta  : 3
GC Time Delta   : 13 ms
Memory Before   : 1150.16 MB
Memory After    : 826.52 MB
Checksum        : 275424
-----
Round           : 2
Execution Time  : 191 ms
Avg Pause Time  : 4.67 ms
Throughput      : 92.67 %
CPU Usage       : 8.14 %

```

5.4 Final Benchmark Summary

Across extended benchmark summaries, we observed sustained throughputs reaching up to 95.07%, with average GC times hovering around 22.27 ms per round, and highly efficient memory reclamation (Avg Mem Before: 378.85 MB -> Avg Mem After: 365.74 MB across the test baseline).

```

-----
Round          : 14
Execution Time  : 717 ms
Avg Pause Time  : 4.20 ms
Throughput      : 97.07 %
CPU Usage       : 3.68 %
GC Count Delta  : 5
GC Time Delta   : 21 ms
Memory Before   : 540.07 MB
Memory After    : 185.10 MB
Checksum        : 275424
-----

Round          : 15
Execution Time  : 306 ms
Avg Pause Time  : 4.75 ms
Throughput      : 93.79 %
CPU Usage       : 6.69 %
GC Count Delta  : 4
GC Time Delta   : 19 ms
Memory Before   : 185.10 MB
Memory After    : 396.22 MB
Checksum        : 275424
=====
FINAL SUMMARY
Average Exec Time : 499.93 ms
Minimum Exec Time : 227 ms
Maximum Exec Time : 947 ms
Avg Pause Time    : 5.08 ms
Throughput        : 95.07 %
CPU Usage         : 4.78 %
Avg GC Time       : 22.27 ms
Avg GC Count      : 4.40
Avg Mem Before    : 378.85 MB
Avg Mem After     : 365.74 MB
=====

```

6. CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

This project successfully analyzed the performance of the JVM's Serial, Parallel, and G1 Garbage Collectors under a memory-intensive 2GB heap workload. The findings validate theoretical expectations:

- **Serial GC** is inefficient for large memory workloads on multi-core systems, evidenced by its lower throughput (~87%).
- **Parallel GC** excels at maximizing throughput, making it ideal for backend batch-processing applications where pause times are a secondary concern.
- **G1 GC** provides the best overall balance for modern server applications, offering throughput comparable to Parallel GC while keeping average pause times managed and predictable.

6.2 Future Scope

- **Testing Next-Gen Collectors:** Future research can expand to include ultra-low-latency collectors like **ZGC (Z Garbage Collector)** and **Shenandoah GC** to evaluate their performance on terabyte-scale heaps.
- **Varying Workloads:** Testing against different allocation profiles (e.g., highly concurrent web server traffic vs. large object caching) to see how GC behavior shifts.
- **Heap Sizing Impacts:** Running the same benchmarks across varying heap sizes (e.g., 512MB vs 8GB vs 32GB) to map the degradation or improvement curves of each algorithm.

REFERENCES

1. Oracle Corporation. (n.d.). *HotSpot Virtual Machine Garbage Collection Tuning Guide*. Java Platform, Standard Edition. Retrieved from
2. Oaks, S. (2020). *Java Performance: In-Depth Advice for Tuning and Programming Java 8, 11, and Beyond*. O'Reilly Media.
3. Hunt, C., & John, M. (n.d.). *Understanding Java Garbage Collection*. Oracle Technology Network.
4. Goetz, B. (2006). *Java Concurrency in Practice*. Addison-Wesley Professional. (For insights on thread management and memory visibility).